DAUST Student Government

# Finance Tracker

## Feasibility Assessment & Open-Item Resolution Report

Scope: the whole product (version 1 + version 1.1), not a pilot. Prepared following documentation research, inspection of the official Naboopay Python SDK, the *yoonpay* reference integration, and live testing against the production Naboopay API.

| | |
|---|---|
| **Document** | Finance Tracker — Technical Brief v1.1 (12 Oct 2026) |
| **Assessment date** | 13 October 2026 |
| **Method** | Naboopay has no public sandbox. Facts were resolved from (a) the official Naboopay Python SDK (its pydantic models are the API contract), (b) *yoonpay*, an open-source gateway run end-to-end against the real Naboopay APIs, and (c) direct live calls to `https://api.naboopay.com` with the provided credentials. |

## Headline verdict

**Overall feasibility: 90 / 100.** The project is highly buildable as specified. The core ledger — computed balances, manual entry, automatic shop/DaBus income capture, labels/inbox/filters/CSV, reconciliation, the end-of-term report, login and audit log — is fully feasible on the suggested Convex + React stack and scores in the mid-90s.

Points are deducted in exactly three places, none of which block the core product: (1) Naboopay exposes **no automatic payout/fee feed**, so part of Milestone 5 must be done by manual entry; (2) **true offline-first** capture is not free on Convex and should be staged; and (3) a working Naboopay **server API key** needed for two optional automations is not yet in hand (the test key returned "Invalid API key" and was revoked during testing).

## Live test results (13 Oct 2026)

| Check | Result |
|---|---|
| **Base URL** | `https://api.naboopay.com/api/v1` is live and correct (the bare host 302-redirects to the marketing site). Same base the proven *yoonpay* integration uses. |
| **Auth scheme** | `Authorization: Bearer <key>`. Confirmed by server behaviour (missing/invalid key → 401). |
| **Webhook secret** | **Verified working.** 64 hex chars = 32-byte key. Reproduced Naboopay's exact signing (HMAC-SHA256 per the SDK) and verified a signature successfully. |
| **Webhook raw-body pitfall** | **Proven empirically.** Signing raw bytes vs. a re-serialized parsed object yields different signatures. Verify against the **raw request body**, never a re-dumped dict. |
| **Server API key** | **Pending a fresh key.** The test key first returned transient 500s, then consistently `401 {"detail":"Invalid API key"}` on every server endpoint (account, list, create). The account owner confirmed the key was **revoked** during testing, which explains the 401. Live verification of create / account / list is therefore pending a fresh server API key; all schema facts in this report are already established from the official SDK. |

Note on the credential: there are two distinct secrets. The **webhook secret** (generated together with the webhook URL in the dashboard) verifies inbound webhook signatures — this is already verified working. The **server API key** (the Bearer token) is what application code uses to call Naboopay endpoints — this is the one that was revoked. Crucially, core income capture does

not depend on the *tracker* holding a server API key: the shop and DaBus backends already own working Naboopay credentials and are the capture point. The tracker needs a read key only for two optional automations (the safety-net sweep and auto-reconciliation of the Naboopay balance).

# Feasibility scorecard — feature by feature

| Feature | Score | Verdict & reason |
|---|---|---|
| **Ledger with live computed balances (Naboopay, Wave, OM, Cash, School)** | **97** | Fully feasible. Append-only ledger + reversal model + Convex live queries give instant, correct balances on every device. FCFA integers fit v.number() safely. |
| **Manual entry (income / expense / transfer / allocation / withdrawal / adjustment), phone-first** | **95** | Fully feasible. Standard React form + Convex mutation; reason-required validation and recent-value suggestions are routine. |
| **Automatic income — shop & DaBus intake endpoint (idempotent)** | **90** | Feasible. Tracker side is a Convex httpAction keyed on (source, externalId). Requires the shop/DaBus backends to persist Naboopay's returned order_id and call the intake on the paid/done transition. |
| **Labels, review inbox, filters, event view, CSV export** | **97** | Fully feasible. Pure application logic over the transactions table. |
| **Reconciliation + audit log** | **93** | Feasible. Manual entry of the real balance always works. Auto-fetching the Naboopay balance (GET /account/) is a bonus that needs a working server API key. |
| **Naboopay feed (Milestone 5): webhook capture, safety-net, payouts, fees** | **65** | Partially feasible. Webhook capture is verified and ready. Safety-net sweep + auto-balance need a working server API key (currently blocked). An automatic payout/fee feed does NOT exist in the API → payouts/fees must be entered manually. |
| **End-of-term review report** | **95** | Fully feasible. Aggregation over the ledger reproduces totals by category/event and start/end balances. |
| **Recurring expense templates (v1.1) — monthly drafts** | **90** | Feasible. A Convex scheduled job creates one draft per template per month; drafts change no balance until confirmed. |
| **Officer login & access control (v1: equal rights)** | **95** | Fully feasible. Convex auth; per-transaction recorder and audit log are straightforward. |
| **Offline queueing (should-have)** | **72** | Feasible but not out-of-box. Convex natively survives short network blips with exactly-once delivery; true offline-first (across restarts) needs a small IndexedDB outbox. Idempotent intake makes replay safe. |
| **Cutover from Excel (verified opening balances)** | **95** | Feasible. A process, not a technical risk; the standard, correct approach for a ledger migration. |

Weighting the core features (≈80% of delivered value, avg ~94), Milestone 5's partial piece (~65), staged offline (~72) and recurring (~90) gives an overall ≈ **90/100**. Core-only feasibility is ≈ **95/100**.

# What is not feasible as originally written — and how to resolve it

| Item | Why | How to resolve |
|---|---|---|
| **Automatic Naboopay payout & fee feed (part of Milestone 5)** | The public API exposes no settlement/payout feed and no fee statement. GET /account/ gives a balance; GET /transaction/get-transactions gives transactions; neither reports Naboopay→Wave/OM payouts or per-transaction fees. | Enter payouts (as transfers) and fees (as expenses) manually — the brief already allows this. Keep fees_customer_side=true so the credited amount equals the nominal amount and no per-sale fee expense is needed at all. |
| **Full offline-first capture (should-have)** | Convex does not provide offline persistence across app restarts out of the box; it only queues across short blips. | Ship v1 on native blip-handling; add a lightweight IndexedDB outbox later that replays on reconnect. Safe because intake is idempotent (rule 7). |
| **Tracker's own live Naboopay automations (safety-net sweep, auto-reconcile balance)** | The test server API key was revoked during testing and now returns "Invalid API key"; no working server key is currently in hand. | Generate a fresh server API key in the dashboard (ensure the account is activated for production). Core income capture is unaffected — it flows through the shop/DaBus backends, which already hold working credentials. The webhook secret is a separate, already-verified credential. |
| **Importing the old Excel history** | Explicitly out of scope by design — the data is too inconsistent and would poison the audit trail. | Not a defect. Start from verified opening balances; optionally add one summary entry per past event so event totals exist in the report. |

# Resolution of the brief's open items (§15)

## Q1. How does Naboopay deliver data — webhook, polling, or export?

**Resolved: webhook (push), with a pull API as backup.**

The SDK ships a real webhook module. Naboopay POSTs a signed event; GET /transaction/get-transactions (list) and GET /transaction/get-one-transaction?order_id= (single) are available to poll.

Webhook payload (WebhookModel) carries exactly: order_id, amount, currency (XOF), created_at, transaction_status. The signature is a hex HMAC-SHA256 of the body using the webhook secret; the callback URL is registered in the dashboard.

**Pitfall:** verify the HMAC against the raw request body bytes, not a re-serialized dict — in Convex use httpAction + request.text() before parsing. (Proven in live testing.)

## Q2. Does the confirmation carry our order/booking reference for matching?

**Resolved: No merchant-reference field — this changes the design.**

TransactionRequest has no merchant-reference / external-id / metadata field. You can send only: method_of_payment, products[] (name, category, amount, quantity, description), success_url, error_url, fees_customer_side, is_escrow, is_merchant. You cannot inject your own order ID.

Instead Naboopay generates order_id and returns it at create; the webhook later echoes the same order_id. **Correction to §6:** the shop/DaBus backend must store Naboopay's returned order_id against its own order, and the tracker's naboopayReference must be that order_id. The §7 safety-net then joins on order_id. A backend that forgets to persist the returned order_id has no way to recover the link (the webhook contains no products and no custom field).

## Q3. Is the credited amount gross or net of fees?

**Resolved: your choice — and the right choice is gross.**

TransactionRequest.fees_customer_side defaults to True. The response exposes both amount (nominal) and amount_to_pay (what the customer is charged).

With fees_customer_side = True the customer pays amount + fee and the SG receives the full nominal amount (the webhook's amount equals what hits the balance). With False, the SG receives amount – fee, but the webhook still reports the nominal amount and does not include the fee — the ledger would overstate every sale.

**Recommendation:** keep fees_customer_side = True; then ledger income = credited amount and no per-sale 'Provider fees' expense is needed for shop/DaBus income.

## Q4. Do payouts and fees appear in retrievable data?

**Resolved: essentially no — the automatic '§7 feed' does not exist as an API.**

There is no settlement/payout feed and no fee statement. Available: GET /account/ (balance, account_number, account_is_activate, method_of_payment, loyalty_credit); GET /transaction/get-transactions (order_id, amount, amount_to_pay, status, created_at); and cashout (Wave/OM) which is a merchant-initiated outgoing payment with no idempotency key and no callback.

**Consequences:** payouts and fees must be entered manually (§7 should be rewritten around manual entry). The safety-net is still achievable by periodically listing transactions and flagging any paid/done order_id with no matching income. Bonus: GET /account/ gives the real Naboopay balance, so §10 reconciliation of that account can be automated once a server API key is available.

## Q5. Where in the shop/DaBus backend should the tracker call go?

**Resolved: at the status-confirmed transition, not at checkout.**

The backend creates a Naboopay transaction (pending), redirects the customer to checkout_url, then receives the webhook / polls get-one-transaction. The tracker intake call belongs only on the transition to paid/done — exactly rule 4 ('one capture point') and §6 step 3.

## Q6. Default categories/events for shop & DaBus.

**Treasurer decision — not a technical blocker.**

Left to the Treasurer as the brief states. Note a new edge case from the status enum: statuses are pending, paid, done, part_paid. Treat part_paid as not-yet-income (surface in Needs-review; post only on paid/done). Naboopay has no refund API — a refund is a manual cashout; the tracker's own refund→reversal is correct, but the real money-out is a separate manual step.

# Other flagged risks

## Offline queueing — feasible, but not free with Convex.

Convex natively queues mutations across brief network blips with exactly-once delivery, but does not provide full offline-first persistence across restarts. Options: (1) rely on native blip-handling and label offline a short-gap convenience (cheapest, matches 'should have'); (2) add a small IndexedDB outbox that persists unsent entries and replays on reconnect (safe, because intake is idempotent); (3) a community local-first layer (overkill for v1). Recommendation: ship v1 with option 1, design the form so option 2 can be added later.

## Cutover / no Excel import — sound as written; one enhancement.

'Start from verified opening balances' is the standard, correct approach for a ledger migration; importing inconsistent data would poison the audit trail. The only gap (pre-cutover event history) is already mitigated by the brief's optional per-event summary entries. Enhancement: make that per-event summary step explicit for any event you want in the review report, and keep the Excel as a read-only archive. No blocker.

# Additional issues surfaced

| # | Issue | Impact | Recommendation |
|---|---|---|---|
| **1** | No Naboopay sandbox — test vs live is only the API key | Milestone 2 acceptance ('same order twice', 'refund creates reversal') cannot be tested without real money | Test with minimal real amounts (e.g. 100 FCFA) and cash out, or test against a mocked intake; budget for this. |
| **2** | Webhook HMAC over raw body | Silent verification failures | Verify raw bytes before parsing (httpAction / request.text()). |
| **3** | cashout has no idempotency/callback | Duplicate real payouts if ever automated | Keep 'app does not move money' (out of scope); never wire auto-payouts. |
| **4** | part_paid status | Mis-posted / duplicated income | Post income only on paid/done; route part_paid to Needs-review. |
| **5** | TransactionResponse.amount defaults to 0 if omitted | Zero-value ghost income | Reject/flag amount ≤ 0 on intake (the brief already requires positive integers — enforce it). |
| **6** | Rate limit 100 req / 60 s | Safety-net polling could trip it | Poll on a schedule with pagination, not per-item. |
| **7** | Convex number types | FCFA are integers (rule 6) | FCFA magnitudes are well under 2^53, so v.number() is safe; use v.int64() to enforce integrality. Use httpAction for /intake/income. |
| **8** | Matching depends on backend persisting order_id | A dropped create-response = unrecoverable link | Persist Naboopay order_id transactionally at create time; reconcile via get-transactions as backstop. |
| **9** | Intermittent Naboopay 5xx observed during testing | Transient outages can drop income callbacks | Make the shop/DaBus → tracker intake retry on failure (the brief requires this); rely on the idempotent (source, externalId) rule. |

# The genuine limits — what we truly cannot do

Separating real dead-ends from things that are merely more work, there are only four genuine limits. Two are hard (the platform cannot do it); two are a data/design constraint. Each has a clean workaround already compatible with the brief — none force a redesign of the core ledger.

### 1. We cannot get payouts or fees from Naboopay automatically. (Hard limit)

**Why:** the API has no endpoint for it. We can read the transaction list, a single transaction, and the current balance — but there is no feed of 'Naboopay settled X to Wave/OM' and no per-transaction fee breakdown. It is not hidden or locked; it does not exist in the API. **How:** enter payouts (as transfers) and fees (as expenses) manually — the brief already allows this — and set fees_customer_side = true so the customer bears the fee and there are no per-sale fees to record at all.

### 2. We cannot test the payment flow without real money. (Hard limit)

**Why:** Naboopay has no sandbox; 'test' vs 'live' is only which key you use, and both move real money. **How:** test Milestone 2 with tiny real amounts (e.g. 100 FCFA) and cash out afterwards, and/or mock the intake endpoint to exercise the tracker's own logic (duplicates, reversals) without touching Naboopay. Budget a small amount for this.

### 3. We cannot reliably import the old Excel history. (Data limit, not technical)

**Why:** the Excel is too inconsistent; importing it would carry errors into a ledger whose entire value is being trustworthy. **How:** start from verified opening balances (already the plan) and optionally add one summary line per past event you want in reports; keep the Excel as a read-only archive.

### 4. We cannot tag a Naboopay payment with our own order reference. (Design constraint)

**Why:** create-transaction has no merchant-reference field. Naboopay generates the order_id and returns it; the webhook echoes only that same order_id. **How:** the shop/DaBus backend must store Naboopay's returned order_id at creation and match on it later. Related: there is no refund API — a refund is a manual cash-out, with the tracker recording a reversal.

Not on this list (just engineering, not 'cannot'): offline entry, the tracker auto-reading the Naboopay balance or running the safety-net sweep (needs only a server key), and all core features — live balances, manual entry, inbox, filters, CSV, reconciliation, report, login, recurring drafts.

# What we will implement

The build we will deliver, in order, with the resolutions above baked in:

| Module | What we build |
|---|---|
| **Milestone 1 — Foundation** | Schema (accounts, transactions, categories, events, reconciliations, users, auditLog, recurringTemplates), officer login, the phone-first manual entry form, and live computed balances via Convex live queries. Balances are opening balance + transactions; reversals use the same formula. |
| **Milestone 2 — Automatic income (shop & DaBus)** | An idempotent intake HTTP endpoint (Convex httpAction) keyed on (source, externalId), called by the backends on the paid/done transition. Each source has its own key; duplicates return the existing record; refunds create a reversal; retries are safe. |
| **Milestone 3 — Review & reporting primitives** | Needs-review inbox (unlabeled items, unmatched payments, recurring drafts), filters (date/account/category/event/source/type), event view, and CSV export. |
| **Milestone 4 — Reconciliation & audit** | Per-account reconciliation against the real balance with adjustment entries; full audit log of who/when/what for every edit. |

| Module | What we build |
|---|---|
| **Milestone 5 — Naboopay integration (rescoped)** | (a) Webhook-confirmed income capture matched by order_id; (b) a scheduled get-transactions safety-net sweep flagging paid/done payments with no matching income; (c) manual entry of payouts and fees; (d) bonus: auto-reconcile the Naboopay balance via GET /account/ once a server key is available. |
| **Milestone 6 — Review report** | Period summary: totals by category and event, start/end balances, School-account movement, and reconciliation history — the report that replaces the end-of-term reconstruction. |
| **Milestone 7 (v1.1) — Recurring expenses** | Templates that generate one draft expense per month; drafts change no balance until confirmed; overdue drafts are visible on the home screen. |
| **Cross-cutting** | Raw-body webhook signature verification; amount > 0 integer enforcement on intake; part_paid handling; offline via Convex's native blip-queue in v1 (IndexedDB outbox staged for later). |

# Design & feature changes vs. the original brief

These are the concrete changes to the brief that our findings require. They are small and localised.

| Area | Change | Reason |
|---|---|---|
| **§6 matching reference** | naboopayReference becomes Naboopay's returned order_id; the shop/DaBus backend must persist it at create time. | create-transaction accepts no merchant reference; order_id is the only link and it is generated by Naboopay. |
| **§7 'Naboopay feed'** | Rewrite from an automatic feed to: webhook capture + scheduled pull safety-net + manual payout/fee entry. | No settlement/payout feed or fee statement exists in the API. |
| **Fees handling** | Set fees_customer_side = true; drop the per-sale 'Provider fees' expense for shop/DaBus income. | Customer-side fees mean the credited amount equals the nominal amount — no fee to subtract or record. |
| **Transaction status** | Add part_paid handling: treat as not-yet-income, show in Needs-review; post income only on paid/done. | Naboopay's status enum includes part_paid, which the brief's binary model did not cover. |
| **Refunds** | Tracker records a reversal; document that the real money-out is a separate manual cash-out. | Naboopay has no refund API. |
| **Offline capture** | Stage it: Convex native blip-queue in v1; add an IndexedDB outbox later for true offline-first. | Convex does not provide offline persistence across restarts out of the box. |
| **Intake validation** | Enforce amount > 0 integers; (source, externalId) idempotency; verify webhook HMAC over the raw body. | Defends against ghost/zero income, duplicates, and silent signature-verification failures. |

# Net effect on the build plan

• **Milestone 5 is unblocked but should be rescoped:** there is no automatic payout/fee feed. Implement it as (a) webhook-confirmed income capture by order_id, (b) a periodic get-transactions safety-net sweep, and (c) manual entry of payouts and fees — plus the bonus of automated Naboopay-balance reconciliation via GET /account/ once a server key is available.

• **The brief's §6/§7 text needs two corrections:** the reference is Naboopay's returned order_id (you cannot send your own), and §7's 'feed' becomes mostly manual plus a pull-based safety net.

• **Credential action:** generate a fresh Naboopay server API key (the Bearer token the app uses to call endpoints) and ensure the account is activated for production. This is separate from the webhook secret, which is already verified.

# What we need from your side — to agree before development

Development can begin on the core ledger immediately. The items below are what we need from the SG / IT Committee side to complete the Naboopay-dependent parts and to lock the scope. Let us agree on these first.

| # | What we need | Specifically | Why |
|---|---|---|---|
| 1 | **Fresh server API key (activated for production)** | A new Bearer API key from the dashboard's API/Developer section, with the account activated and Wave/OM enabled. | Enables the safety-net sweep and auto-reconciliation of the Naboopay balance. Core capture does not depend on it. |
| 2 | **Webhook registration** | Register our endpoint URL (a Convex httpAction, e.g. https://<deployment>.convex.site/intake/naboopay), and confirm the exact signature header name and whether regenerating the secret is required. | So Naboopay delivers signed events to us and we verify them correctly. |
| 3 | **Shop & DaBus backend changes + access** | The backends must call our intake on the paid/done transition and persist + send Naboopay's order_id. We need to coordinate with whoever maintains them. | They are the single capture point for automatic income (rule 4); the order_id is the only match key. |
| 4 | **Fee policy decision** | Confirm fees_customer_side = true (the customer bears the Naboopay fee). | Keeps ledger amount = credited amount and removes per-sale fee bookkeeping. |
| 5 | **Default categories & events (Treasurer)** | Agree the starting category list and the default category/event the shop and DaBus attach to their sales. | Automatic income needs a category; manual entry needs an agreed list. |
| 6 | **Cutover inputs** | A cutover date; verified opening balances (Naboopay/Wave/OM apps + a physical cash count); a university statement for the School balance. | The app starts from reality, not the Excel; first reconciliations confirm the starting point. |
| 7 | **Offline scope decision** | Accept staged offline for v1 (native blip-handling now, full offline-first later)? | Full offline-first is extra work; we want agreement before committing effort. |
| 8 | **Infrastructure & branding** | GitHub org + hosting location; branding assets (IBM Plex Sans, SG logos). | Code lives in the IT Committee org; look must match the other SG tools. |
| 9 | **Small test budget** | A small amount of real money (e.g. a few hundred FCFA) for live testing. | Naboopay has no sandbox; Milestone 2 acceptance needs real transactions. |

## The two Naboopay secrets — do not confuse them

| Secret | Generated where | Used for | Status |
|---|---|---|---|
| **Webhook secret** | With the webhook URL in the dashboard | Verifying inbound webhook signatures (HMAC-SHA256 over the raw body) | **Verified working** |
| **Server API key (Bearer)** | API / Developer section of the dashboard | Calling Naboopay endpoints: create-transaction, account, get-transactions | **Revoked — need a fresh one** |

# Sources


- Finance Tracker — Technical Brief v1.1 (12 Oct 2026)
- Naboopay API docs — https://docs.naboopay.com/
- Naboopay official Python SDK — https://github.com/naboopay/naboopay-python-sdk (models.py, webhook.py)
- yoonpay reference integration — https://github.com/crossben/yoonpay (NabooPayProvider.java, docs/providers/naboopay.md)

- Live calls to https://api.naboopay.com/api/v1 (13 Oct 2026): auth, webhook HMAC, key-rejection confirmed
- Convex — offline/queued mutations & sync platform (stack.convex.dev, convex.dev/sync)